Check for updates

ACM DIGITAL LIBRARY | Association for Computing Machinery | acm open

Latest updates: https://dl.acm.org/doi/10.1145/3746252.3761116

RESEARCH-ARTICLE

Beyond Return Conditioning: Multi-Scale Sequence Modeling and Advantage-Guided Policy Routing for Offline RL

KUNBAO WU, Beijing University of Posts and Telecommunications, Beijing, Beijing, China

XINNING ZHU, Beijing University of Posts and Telecommunications, Beijing, Beijing, China

YANG QIN, Beijing University of Posts and Telecommunications, Beijing, Beijing, China

TIERU WANG, Beijing University of Posts and Telecommunications, Beijing, Beijing, China

JIANZHOU DIAO, Beijing University of Posts and Telecommunications, Beijing, Beijing, China

ZHENG HU, Beijing University of Posts and Telecommunications, Beijing, Beijing, China

Open Access Support provided by:

Beijing University of Posts and Telecommunications

PDF Download
3746252.3761116.pdf
11 January 2026
Total Citations: 0
Total Downloads: 83

Published: 10 November 2025

Citation in BibTeX format

CIKM '25: The 34th ACM International Conference on Information and Knowledge Management
November 10 - 14, 2025
Seoul, Republic of Korea

Conference Sponsors:
SIGWEB
SIGIR

https://doi.org/10.1145/3746252.3761116
ISBN: 9798400720406

Beyond Return Conditioning: Multi-Scale Sequence Modeling and Advantage-Guided Policy Routing for Offline RL

Kunbao Wu
The State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications
Beijing, Beijing, China
wukb@bupt.edu.cn

Xinning Zhu
Beijing University of Posts and Telecommunications
Beijing, Beijing, China
zhuxn@bupt.edu.cn

Yang Qin
Beijing University of Posts and Telecommunications
Beijing, Beijing, China
qinyoung@bupt.edu.cn

Tieru Wang
Beijing University of Posts and Telecommunications
Beijing, Beijing, China
wtr_2583561398@bupt.edu.cn

Jianzhou Diao
Beijing University of Posts and Telecommunications
Beijing, Beijing, China
djz@bupt.edu.cn

Zheng Hu*
The State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications
Beijing, Beijing, China
huzheng@bupt.edu.cn

Abstract

Return-conditioned supervised learning (RCSL) in offline reinforcement learning (RL) leverages Transformers to extract behavioral patterns from offline datasets for decision-making. However, it suffers from inherent limitations in comprehensively capturing multi-scale temporal relationships in historical trajectories. Moreover, its return-conditioning mechanism offers limited guidance in exploiting high-quality behavioral patterns, often resulting in suboptimal action generation during inference. To address these challenges, we propose the **A**dvantage **D**ecision **C**onv**M**amba (ADCM), a method that integrates multi-scale sequence modeling (MSSM) with advantage policy guidance (APG). ADCM reconstructs historical sequences through patch partitioning and employs Mamba architecture together with causal convolutions to model sparse global dependencies and dense local Markovian dependencies for behavioral pattern discovery. By incorporating relative advantage action sampling based on the Mixture-of-Experts (MoE) framework, ADCM prioritizes high-quality actions during inference, thereby reducing reliance on low-quality behavioral patterns in the dataset. We evaluate ADCM on multiple offline RL benchmarks from D4RL. Experimental results show that ADCM achieves significant improvements over baseline models, with particularly strong performance on suboptimal datasets. The code for ADCM is available at https://github.com/iTom233/ADCM.git.

*Corresponding Author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CIKM '25, Seoul, Republic of Korea.
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2040-6/2025/11
https://doi.org/10.1145/3746252.3761116

CCS Concepts

• **Computing methodologies** → **Sequential decision making**; *Supervised learning*; Neural networks.

Keywords

reinforcement learning; sequence modeling; advantage policy

ACM Reference Format:
Kunbao Wu, Xinning Zhu, Yang Qin, Tieru Wang, Jianzhou Diao, and Zheng Hu. 2025. Beyond Return Conditioning: Multi-Scale Sequence Modeling and Advantage-Guided Policy Routing for Offline RL. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25), November 10–14, 2025, Seoul, Republic of Korea.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3746252.3761116

1 Introduction

Offline reinforcement learning (RL) aims to learn policies from static datasets without real-time interaction [7, 8, 17, 18, 26], and has demonstrated strong potential in robotics [12, 20], language generation [11, 25], and autonomous driving [19, 23]. In recent years, Transformer-based decision models have turned the offline policy learning problem into a conditional autoregressive action prediction problem [2, 5, 9]. These methods utilize the attention mechanism to model historical sequences and guide the policy to predict actions through return values. We call this paradigm return-conditioned supervised learning (RCSL). However, as illustrated in Figure 1, two issues need to be optimized for the RCSL.

On the one hand, the original Transformer architecture struggles to comprehensively model long-term dependencies and local Markovian dependencies in the RL sequence, thereby failing to explore the behavioral patterns within offline datasets. To address this weakness, prior works [16, 21, 27] have attempted to enhance sequence modeling ability by using the Convolutional Neural Network (CNN) to replace the attention mechanism for Markovian dependency modeling, or adopting a dual-branch structure for sequence modeling. However, these methods overlook the fact that

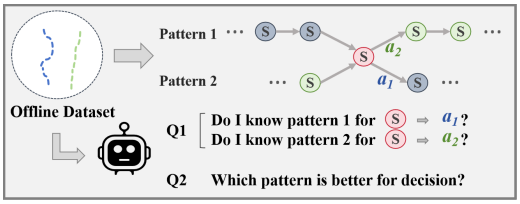


Figure 1: Conceptual issues of RCSL. The same state may correspond to different actions across trajectories. Accurately distinguishing these patterns depends on effective sequence modeling, while determining the more advantageous behavior requires robust value guidance.

the misalignment among the representation subspaces of returns, states, and actions leads to scale discrepancies between long-term dependencies and local Markovian dependencies. This restricts the ability of existing methods to represent different behavioral patterns in the sequence. Therefore, a fundamental research question arises:

Question 1: *How can we design a more effective sequence modeling approach that enables RCSL to better extract behavioral patterns from datasets?*

On the other hand, RCSL inherits the simplicity of supervised learning but heavily depends on the quality of the dataset. Although return conditioning can provide guidance on behavioral quality, its validity assumption is more stringent compared to dynamic programming [1, 22], making RCSL prone to generate suboptimal actions in complex and dynamic inference scenarios. To address this weakness, some works [28, 31] have introduced Q-learning to train policies that force the output of high Q-value actions, rather than fitting the behavioral patterns in the dataset. However, existing approaches struggle to balance the conflicting optimization objectives between supervised learning and maximizing the value of actions during training, which not only restricts the potential of RCSL in behavioral patterns mining but also undermines its simplicity and accuracy in policy learning. Therefore, another crucial question of RCSL is:

Question 2: *How can we guide the RCSL to predict the high-quality action based on behavioral patterns within the dataset?*

To address the above questions, we propose **A**dvantage **D**ecision **C**onv**M**amba (ADCM), a novel RCSL architecture that integrates multi-scale sequence modeling and advantage policy guidance. For Question 1, we reconstruct trajectories via patch partitioning, leverage Mamba architecture to capture sparse long-range dependencies across patches, and apply causal convolution to extract dense Markovian dependencies within each patch. Finally, multi-scale dependencies are dynamically fused to learn contextual representations of diverse behavioral patterns within the dataset. For Question 2, we introduce multiple expert networks that predict actions based on context-aware state representations and specialize in different behavioral distributions through a gating mechanism during training. In inference, we sample the relatively advantageous action from all expert outputs via a pre-trained critic, thereby achieving high-quality decision-making.

The contributions of this paper are summarized as follows:

- We propose a novel sequence modeling framework tailored for RCSL. It leverages patch partitioning to capture multi-scale dependencies, enabling the discovery of diverse behavioral patterns in RL datasets.
- To the best of our knowledge, we are the first to incorporate a Mixture-of-Experts (MoE) mechanism into RCSL for policy guidance. It maintains the simplicity of policy learning while fully leveraging the discovered behavioral patterns to achieve high-quality decision-making.
- We conduct comprehensive evaluations on 12 datasets from Gym and Maze2d tasks, demonstrating the effectiveness of ADCM across diverse scenarios.

2 Related Work

2.1 Sequence Modeling for Offline RL

Offline RL learns the policy directly from static datasets, avoiding costly or risky environment interactions. Early value-based methods [7, 8, 15, 18, 29, 32] update policy using Q-values and mitigate distributional shift by approximating the behavioral patterns in the dataset through policy constraints or Q-value regularization. More recently, RCSL methods [2, 14, 24] reformulate trajectories as sequences of states, actions, and returns, enabling autoregressive action prediction via the Transformer. This paradigm simplifies policy learning and alleviates distributional shift issues in traditional value-based methods. Some researchers argue that the attention mechanism of the Decision Transformer (DT) [2] just focuses on capturing the coarse-grained dependencies within the sequence and ignores the other dependencies within the sequence. In response to this, Graph Decision Transformer (GDT) [13] models the input sequence as a causal graph to capture the potential dependencies between fundamentally different concepts. Decision ConvFormer (DC) [16] proposes using causal convolution to replace the attention mechanism, which focuses on capturing the Markovian dependency. Decision Mamba (DM) [21] and Long-Short Decision Transformer (LSDT) [27] adopt a dual-branch modeling structure to capture the global and local dependencies, which obtain state representations with more comprehensive historical information to enhance conditional behavior imitation. However, these approaches overlook the scale discrepancy between global dependencies and local Markovian dependencies, which arises from the misalignment of multimodal representation subspaces.

2.2 Policy Guidance for RCSL

RCSL leverages future return statistics of the state to learn the behavior distribution from datasets. During inference, the policy generates actions that both align with the behavior distribution and meet the desired return by conditioning on a specified target return. DT and its variants [2, 16, 27] use cumulative rewards as the condition signal to guide the policy. RvS [5] introduces future states and average returns as condition signals to guide policy, and systematically investigates the policy guidance capabilities of different conditional inputs across various environments. Such hindsight-conditioned policy guidance approaches rely on strong determinism assumptions, making it difficult to generalize effectively in complex and stochastic environments [1, 22]. Hierarchical

Decision Transformer (HDT) [3] introduces a hierarchical architecture that dynamically extracts sub-goals from context as substitutes for returns, thereby guiding policy action prediction and substantially improving the robustness of policy guidance. Elastic Decision Transformer (EDT) [30] attempts to dynamically adjust the context length during inference, enhancing the generalization of return-conditioned guidance in complex and non-stationary environments. These methods require the additional design of a complex feature representation learning objective. Q-learning Decision Transformer (QDT) [31] uses the Q-function to adjust the return in trajectories and optimizes the policy by training on the corrected dataset. Critic-Guided Decision Transformer (CGDT) [28] adds a Q-value regularization term into the supervised loss in training, forcing the generation of high Q-value actions. However, these methods undermine the accuracy and simplicity of supervised learning.

3 Preliminaries

3.1 Problem Definition

Given a dataset $\mathcal{D}$ of trajectories $\boldsymbol{\tau} = \{(s_0, a_0, r_0), \ldots, (s_T, a_T, r_T)\}$ of state $s_t \in \mathcal{S}$, action $a_t \in \mathcal{A}$ and reward $r_t \in \mathcal{R}$ which is sampled by a behavior policy π_β interacting with a finite horizon Markov Decision Process with horizon T. The goal of RCSL is to learn a policy $\pi(\theta)$ that maximizes the expected return $E\left[R(\boldsymbol{\tau})\right] = E\left[\sum_{t=0}^{T} r_t\right]$. We define $R_t = \sum_{i=t}^{T} r_i$ as the cumulative rewards at timestep t of trajectory $\boldsymbol{\tau}$, referred to as the return to go (RTG), and $\boldsymbol{\tau}_{i:j} := (R_t, s_t, a_t)_{t=i}^{j}$ denotes a sub-trajectory with RTG. The objective of RCSL is typically to minimize the empirical negative log-likelihood loss (NLL), defined as:

$$\mathcal{L}(\theta) = \mathbb{E}_{\boldsymbol{\tau}\sim\mathcal{D}}\left[-\sum_{0\le t<T} \log\left(\pi_\theta\left(a_t \middle| \tau_{0:t-1}, R_t, s_t\right)\right)\right] \tag{1}$$

During inference, the RTG is initialized with a manually specified target return, and is updated at each timestep according to the rule $R_{t+1} = R_t - r_t$.

3.2 Decision Transformer for RCSL

Decision Transformer (DT) [2] casts offline RL as conditional sequence modeling. Given a historical sub-trajectory $\boldsymbol{\tau}_{t-K+1:t-1}$, state s_t, and RTG R_t, DT maps them into a shared embedding space of dimension d, and arranges them in temporal order to form a sequence representation:

$$\boldsymbol{\tau}^e = \{\boldsymbol{e}^R_{t-K+1}, \boldsymbol{e}^s_{t-K+1}, \boldsymbol{e}^a_{t-K+1}, \ldots, \boldsymbol{e}^R_t, \boldsymbol{e}^s_t\} \in \mathbb{R}^{(3K-1)\times d} \tag{2}$$

$$\boldsymbol{e}^R_i = \text{Linear}_R(R_i), \quad \boldsymbol{e}^s_i = \text{Linear}_s(s_i), \quad \boldsymbol{e}^a_i = \text{Linear}_a(a_i) \tag{3}$$

Causal self-attention is then applied over this sequence with a context of length K to predict the current action a_t:

$$a_t = \left(\text{softmax}\left(\frac{\left(\boldsymbol{\tau}^e\mathbf{W}^Q\right)\left(\boldsymbol{\tau}^e\mathbf{W}^K\right)^\top}{\sqrt{d_k}}\right)\left(\boldsymbol{\tau}^e\mathbf{W}^V\right)\right)_{3K-1}\mathbf{W} + \mathbf{b} \tag{4}$$

where $(\cdot)_{3K-1}$ denotes the representation of the last token, which is projected to predict a_t.

3.3 Mamba for Sequence Modeling

Mamba [4, 10] is a recently proposed sequence model that has demonstrated strong performance on long-sequence tasks. Its core component, the selective state space model (S3M), provides dynamic memory by explicitly modeling temporal state transitions. Given a discrete-time sequence $\boldsymbol{x} \in \mathbb{R}^{l\times d}$, S3M produces the output according to:

$$\boldsymbol{h}_k = \bar{\boldsymbol{A}}_k\boldsymbol{h}_{k-1} + \bar{\boldsymbol{B}}_k\boldsymbol{x}_k \tag{5}$$

$$\boldsymbol{y}_k = \boldsymbol{C}_k^\top\boldsymbol{h}_k \tag{6}$$

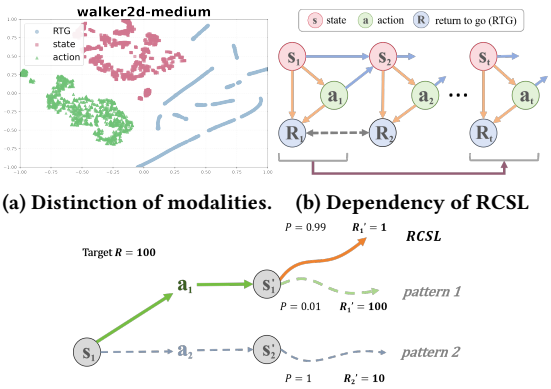
where $\boldsymbol{h}_k \in \mathbb{R}^n$ is the hidden state, $\bar{\boldsymbol{A}}_k \in \mathbb{R}^{n\times n}$ is the state transition matrix, and $\bar{\boldsymbol{B}}_k \in \mathbb{R}^{n\times d}$ and $\boldsymbol{C}_k \in \mathbb{R}^{n\times d}$ are the input and output projection matrices, respectively.

By eliminating hidden states, we obtain an explicit input–output mapping:

$$\boldsymbol{y}_k = \sum_{i=0}^{k} \boldsymbol{C}_k^\top\left(\prod_{j=i+1}^{k}\bar{\boldsymbol{A}}_j\right)\bar{\boldsymbol{B}}_i\boldsymbol{x}_i \tag{7}$$

$$\boldsymbol{y} = \text{S3M}(\bar{\boldsymbol{A}}, \bar{\boldsymbol{B}}, \boldsymbol{C})(\boldsymbol{x}) \tag{8}$$

Here, $\bar{\boldsymbol{A}}, \bar{\boldsymbol{B}}, \boldsymbol{C}$ are input-dependent and govern the model's ability to selectively retain historical information.



(a) Distinction of modalities. (b) Dependency of RCSL

(c) Limitation of hindsight.

Figure 2: Motivation of ADCM. (a) t-SNE visualization shows clear distinctions in the embeddings of three modalities when DT achieves peak performance. (b) Blue arrows denote Markovian transitions, yellow denote causal relationships, gray denote correlations, and purple denote long-term dependencies. (c) The green trajectory represents a high-return but fortuitous behavior pattern, while the blue trajectory represents a moderate-return and stable behavior pattern. Under high-return conditions, RCSL often overlooks the advantages of the latter and mistakenly favors the former's suboptimal actions.

4 Motivation

As shown in Equation (2), the sequence $\boldsymbol{\tau}^e$ consists of three distinct modality embedding vectors (RTGs, states, and actions). These vectors inherently carry distinct semantic meanings and therefore

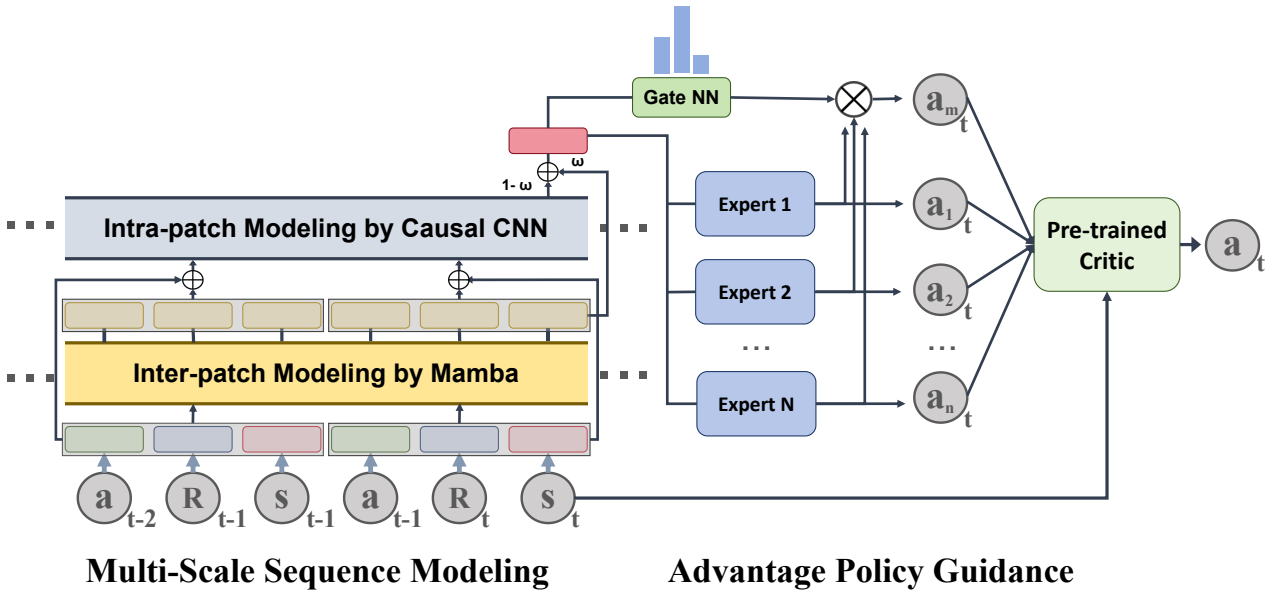


Figure 3: Overview of ADCM. (1) Multi-Scale Sequence Modeling: The input sequence is divided into patches. Mamba captures long-range dependencies across patches, followed by causal convolution for local dependencies within each patch. Feature fusion yields the context-aware state. (2) Advantage Policy Guidance: During training, a gating network assigns context-aware state representations to different experts for fitting diverse behavior distributions. In inference, the policy refers to the action suggestions from all experts and samples the relatively advantageous action via a pre-trained critic.

naturally reside in non-congruent representation subspaces, as illustrated in Figure 2a. Nevertheless, these vectors are arranged with strict periodicity, giving rise to two coexisting dependency patterns, as illustrated in Figure 2b: (1) long-range dependencies across different periods between vectors of the same modality, exhibiting global influence; and (2) local Markovian dependencies between modality vectors within adjacent periods, demonstrating localized influence. Direct modeling of the embedded sequence struggles to differentiate dependencies across temporal scales, resulting in blurred boundaries between global and local dependencies. This motivates us to consider modality distinctions when modeling historical trajectories in offline RL by decoupling dependencies across different scales, thereby achieving a more precise representation.

Although sequence modeling can help RCSL learn diverse behavioral patterns from offline datasets, the decision-making objective during inference is to maximize the overall return, which requires guiding the policy to output actions under high-quality behavioral patterns. The use of RTG as a guiding signal for high-return action generation relies on a deterministic assumption [1, 22], which may mislead the policy in complex and dynamic environments. Specifically, as illustrated in Figure 2c, the policy may generate suboptimal actions based on incorrect behavioral patterns, while other potentially more advantageous patterns fail to influence the decision-making process. Inspired by this, we aim to develop a policy guidance mechanism that can extract advantageous actions from the learned behavioral patterns during inference, thereby avoiding the potential misguidance of RTG.

5 Method

In this section, we focus on two key innovations. First, we propose a multi-scale sequence modeling (MSSM) module that organizes RL trajectories into structured patches to simultaneously capture long-range dependencies and local Markovian dynamics. Second, we present an advantage policy guidance (APG) mechanism, which selects the advantageous action from diverse behavioral patterns during inference. An overview of our proposed method is illustrated in Figure 3.

5.1 Multi-Scale Sequence Modeling

Our MSSM method adopts a patch-based processing strategy to decouple long-term dependencies from local Markovian dynamics. By separately modeling intra-patch and inter-patch dependencies, we construct multi-scale contextual representations to enhance the accuracy of behavioral pattern recognition.

Inter-patch modeling. Given an embedded sequence $\boldsymbol{\tau}^e \in \mathbb{R}^{(3K-1)\times d}$ as described in Equation (2), the sequence is divided into a series of triplet patches (each consisting of three consecutive modality embeddings) using a sliding window with a stride of 1:

$$\boldsymbol{p} = \{\ldots, [\boldsymbol{e}^R{}_i, \boldsymbol{e}^s{}_i, \boldsymbol{e}^a{}_i], [\boldsymbol{e}^s{}_i, \boldsymbol{e}^a{}_i, \boldsymbol{e}^R{}_{i+1}], \ldots\} \tag{9}$$

The patch sequence $\boldsymbol{p} \in \mathbb{R}^{(3K-1)\times 3\times d}$ is first projected to a lower-dimensional space, and then partitioned into three specific sequences $\boldsymbol{p}^{saR} \in \mathbb{R}^{K\times d}$, $\boldsymbol{p}^{aRs} \in \mathbb{R}^{K\times d}$, and $\boldsymbol{p}^{Rsa} \in \mathbb{R}^{(K-1)\times d}$ as

follows:

$$\boldsymbol{p}^{saR}{}_i = \text{Linear}([\boldsymbol{e}^s{}_{i-1}; \boldsymbol{e}^a{}_{i-1}; \boldsymbol{e}^R{}_i]) \in \mathbb{R}^d \tag{10}$$

$$\boldsymbol{p}^{aRs}{}_i = \text{Linear}([\boldsymbol{e}^a{}_{i-1}; \boldsymbol{e}^R{}_i; \boldsymbol{e}^s{}_i]) \in \mathbb{R}^d \tag{11}$$

$$\boldsymbol{p}^{Rsa}{}_i = \text{Linear}([\boldsymbol{e}^R{}_i; \boldsymbol{e}^s{}_i; \boldsymbol{e}^a{}_i]) \in \mathbb{R}^d \tag{12}$$

where $[;]$ denotes the concatenation operator.

Inspired by the selective state space model [4, 10], the attention mechanism is replaced with three independent Mamba modules to selectively extract critical long-term dependencies, thereby generating the long-term feature representation sequence $\boldsymbol{z}^{LT} \in \mathbb{R}^{(3K-1)\times d}$:

$$\boldsymbol{z}^{LT} = \left\{\ldots, \text{S3M}_R(\boldsymbol{p}^{saR})_i, \text{S3M}_s(\boldsymbol{p}^{aRs})_i, \text{S3M}_a(\boldsymbol{p}^{Rsa})_i, \ldots\right\} \tag{13}$$

Here, $\text{S3M}(\cdot)$ denotes the theoretical expression of Mamba, implemented as in Equations (5) and (6).

Intra-patch modeling. Directly modeling intra-patch dependencies on the original embedding sequence may neglect the influence of long-term dependencies on local dynamics. To address this issue, a hierarchical modeling strategy is adopted. Specifically, $\boldsymbol{z}^{LT}$ is added to $\boldsymbol{\tau}^e$ to obtain an enhanced representation sequence $\boldsymbol{x} = \boldsymbol{\tau}^e + \boldsymbol{z}^{LT}$. Then, three separate causal convolution layers are applied to $\boldsymbol{x}$ for intra-patch modeling, capturing modality-specific dependencies across RTGs, states, and actions, and producing a local feature representation sequence $\boldsymbol{z}^{ST} \in \mathbb{R}^{(3K-1)\times d}$ as follows:

$$\boldsymbol{z}^{ST}_{t,j} = \begin{cases} \sum_{l=0}^{L-1} \boldsymbol{w}^R_j[l] \cdot \boldsymbol{x}_{t-l,j} \text{ if } \mod(t,3) = 0 \\ \sum_{l=0}^{L-1} \boldsymbol{w}^s_j[l] \cdot \boldsymbol{x}_{t-l,j} \text{ if } \mod(t,3) = 1 \\ \sum_{l=0}^{L-1} \boldsymbol{w}^a_j[l] \cdot \boldsymbol{x}_{t-l,j} \text{ if } \mod(t,3) = 2 \end{cases} \tag{14}$$

where $\boldsymbol{w}^R_j, \boldsymbol{w}^s_j$, and $\boldsymbol{w}^a_j \in \mathbb{R}^L$ denote the convolutional kernel weights applied to the j-th dimension of $\boldsymbol{x}$, and L denotes the kernel receptive field, corresponding to the patch length.

Feature fusion. We adaptively fuse the two types of feature representation sequences, $\boldsymbol{z}^{LT}$ and $\boldsymbol{z}^{ST}$, to obtain the final multi-scale contextual representation sequence $\boldsymbol{z}$ with a learnable weight $w \in \mathbb{R}$:

$$\boldsymbol{z} = w \cdot \text{LN}(\boldsymbol{z}^{LT}) + (1-w) \cdot \text{LN}(\boldsymbol{z}^{ST}) \in \mathbb{R}^{(3K-1)\times d} \tag{15}$$

where $\text{LN}(\cdot)$ denotes layer normalization. This fusion mechanism enhances the model's adaptability to complex and diverse behavioral patterns.

5.2 Advantage Policy Guidance

To mitigate the potential misguidance of the RTG, we propose an advantage policy guidance mechanism via the MoE framework. Our method yields improvements in both the training and inference stages.

Architecture. As shown in Figure 3, we introduce a MoE framework [33] after the MSSM module, replacing the single action prediction network used in conventional RCSL. We believe that during training, the expert networks can specialize in learning different behavioral patterns. This enables them to provide more diverse behavioral suggestions for sampling the advantageous action during inference.

Training. A small batch of sub-sequences is sampled from the offline dataset and passed through the MSSM module to obtain contextual representations $\boldsymbol{Z} \in \mathbb{R}^{B\times(3K-1)\times d}$. Each contextual state representation $\boldsymbol{z}^s_t$ is fed into the gating network to determine the assigned experts and their corresponding weights:

$$g_i(\boldsymbol{z}^s_t) = \frac{\exp\big((W_r \boldsymbol{z}^s_t)_i\big)}{\sum_{j=1}^N \exp\big((W_r \boldsymbol{z}^s_t)_j\big)}, \quad \boldsymbol{\mathcal{T}}_{\boldsymbol{z}^s_t} = \text{Top-}K\big(\boldsymbol{g}(\boldsymbol{z}^s_t)\big) \tag{16}$$

where $W_r \in \mathbb{R}^{N\times d}$ is the learnable matrix, N is the number of experts, $g_i(\boldsymbol{z}^s_t)$ is the normalized gating probability, and $\boldsymbol{\mathcal{T}}_{\boldsymbol{z}^s_t}$ denotes the indices of the top-K experts (top-2 in our experiments) assigned to $\boldsymbol{z}^s_t$.

Based on $\boldsymbol{\mathcal{T}}$ and $\boldsymbol{g}$, different experts are selected to predict actions for varying states:

$$\hat{a}_t = \text{Linear}\left(\sum_{i\in\boldsymbol{\mathcal{T}}_{\boldsymbol{z}^s_t}} g_i(\boldsymbol{z}^s_t) E_i(\boldsymbol{z}^s_t)\right) \tag{17}$$

where $E_i(\boldsymbol{z}^s_t)$ denotes the output of the i-th expert.

The supervised learning objective is defined as the Mean Squared Error (MSE) between predicted actions and ground-truth actions, consistent with Equation (1). In addition, a load-balancing loss term is introduced to prevent only a few experts from being trained. The final training loss of the policy is:

$$\mathcal{L}_\pi(\theta) = \mathbb{E}_{\boldsymbol{\tau}\sim\mathcal{D}}\Bigg[\underbrace{\frac{1}{K}\sum_{j=0}^{K-1}\|\hat{a}_j - a_j\|^2}_{\text{BC}} + \underbrace{\alpha\, N \sum_{i=1}^{N} f_i G_i}_{\text{LOAD}}\Bigg] \tag{18}$$

Algorithm 1: Advantage Policy Guidance of ADCM

Input: Offline dataset $\mathcal{D}$, critic Q_ϕ, policy π_θ, context length K, horizon T, training steps M_p and M_c

Training

```
// Policy learning
```

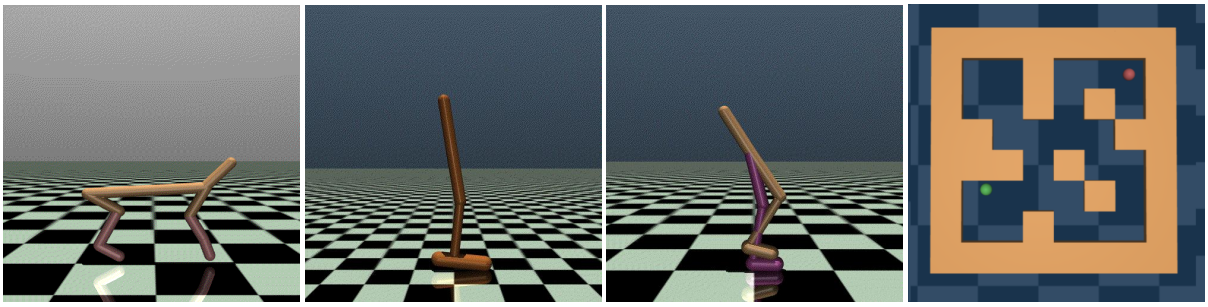
for $i = 1$ **to** M_p **do**
- Sample a batch of length-K sub-trajectories $\{\boldsymbol{\tau}_{t-K+1:t}\}$ from $\mathcal{D}$
- Predict actions using Eq. (17): $\{\hat{a}_{t-K+1}, \ldots, \hat{a}_t\} \sim \pi_\theta(\boldsymbol{\tau}_{t-K+1:t-1}, R_t, s_t)$
- Update π_θ using Eq. (18): $\theta \leftarrow \theta - \eta_\pi \mathbb{E}_{\tau\sim\mathcal{D}}[\nabla_\theta \mathcal{L}_\pi(\theta)]$

```
// Critic learning
```

for $j = 1$ **to** M_c **do**
- Sample a batch of transition tuples (s, a, r, s') from $\mathcal{D}$
- Update Q_ϕ using Eqs. (20) and (21): $\phi \leftarrow \phi - \eta_Q \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\nabla_\phi \mathcal{L}_Q(\phi)\right]$

Inference

for $t = 1$ **to** T **do**
- Input $\boldsymbol{\tau}_{t-K+1:t-1}, R_t, s_t$
- Predict actions using Eqs. (17) and (19): $\{a^m_t, \boldsymbol{A}^E_t\} \sim \pi_\theta(\boldsymbol{\tau}_{t-K+1:t-1}, R_t, s_t)$
- Sample the advantageous action a^{adv}_t via Eqs. (22) and (23)
- Output $a_t = a^{adv}_t$



(a) HalfCheetah (b) Hopper (c) Walker2d (d) Maze2d

Figure 4: Overview of the environments. The first three are continuous control tasks with different robot morphologies, requiring agents to control joint torques for forward locomotion. The last is a maze navigation task, where the agent applies forces to drive a ball toward a target position.

where $f_i = \frac{1}{|\mathcal{B}|}\sum_{z_t^s \in \mathcal{B}} \mathbf{1}\{\arg\max \boldsymbol{g}(z_t^s) = i\}$ is the fraction of tokens assigned to the i-th expert, $G_i = \frac{1}{|\mathcal{B}|}\sum_{z_t^s \in \mathcal{B}} g_i(z_t^s)$ is the average gating probability for expert i, and α is the load-balancing coefficient.

Inference. For the contextual representation z_t^s of the current state s_t output by MSSM, we input it into the MoE module to obtain a main action a_t^m via Equation (17), which reflects the behavioral pattern associated with the target RTG. Additionally, we bypass the gating network and assign z_t^s to all experts, obtaining N alternative actions $\boldsymbol{A}_t^E = \{a_t^0, \ldots, a_t^{N-1}\}$, which reflect alternative behavioral patterns:

$$a_t^i = \text{Linear}\big(E_i(z_t^s)\big) \tag{19}$$

To sample the relatively advantageous action, we require a critic to distinguish and quantify the quality of actions. Inspired by IQL [17], we pre-train a Q-function on the dataset using expectile regression:

$$\mathcal{L}_Q(\phi) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\Big(r(s,a) + \gamma V_\psi(s') - Q_\phi(s,a)\Big)^2\right] \tag{20}$$

$$\mathcal{L}_V(\psi) = \mathbb{E}_{(s,a)\sim\mathcal{D}}\left[L_2^\epsilon\Big(Q_{\hat{\phi}}(s,a) - V_\psi(s)\Big)\right] \tag{21}$$

where $V_\psi(s)$ represents the upper expectile value of the state, which is used for the implicit update of the Q-function $Q_\phi(s,a)$. The loss $L_2^\epsilon(u) = |\epsilon - \mathbf{1}(u<0)|u^2$ ensures that when $\epsilon > 0.5$, the Q-function becomes more sensitive to high-value actions.

We then use the Q-function to evaluate the value of the main action $Q(s_t, a_t^m)$ and the values of the alternative actions $Q(s_t, a_t^i)$ for $a_t^i \in \boldsymbol{A}_t^E$, and approximate the distribution of action values as:

$$Q(s_t, a) \sim \mathcal{N}(\mu_t, \sigma_t^2) \tag{22}$$

where μ_t and σ_t denote the mean and standard deviation of values corresponding to the alternative actions $\boldsymbol{A}_t^E$.

Based on this distribution, the advantageous action is sampled as the final decision output:

$$a_t^{adv} = \begin{cases} a_t^m & Q(s_t, a_t^m) > (\mu_t + \lambda\sigma_t) \\ \arg\max\limits_{a_t^i \in \boldsymbol{A}_t^E} Q(s_t, a_t^i) & \text{otherwise} \end{cases} \tag{23}$$

In this way, the policy maximizes action advantage while maintaining decision-making stability, where λ adjusts the trade-off between conservativeness and radicalness. The detailed procedure of APG is presented in Algorithm 1.

6 Experiments

In this section, we conduct experiments on a range of RL benchmarks to investigate the following research questions:

- **RQ1:** How does ADCM perform compared to other state-of-the-art (SOTA) baselines on offline RL benchmarks?
- **RQ2:** What is the contribution of each component in ADCM?
- **RQ3:** Can ADCM capture complex dependencies in offline RL sequences to mine behavioral patterns?
- **RQ4:** Can ADCM guide the generation of advantageous actions using behavioral patterns from the dataset?

6.1 Dataset and Baseline

Dataset. We evaluate ADCM on D4RL [6], a widely used benchmark suite for offline RL that features a diverse range of tasks. Specifically, we consider two categories of environments: (1) Gym continuous control tasks with Markovian dynamics, including Hopper, HalfCheetah, and Walker2d, each with dataset variants such as medium, medium-replay, and medium-expert; and (2) Maze2d goal-navigation tasks characterized by non-Markovian dynamics, which include umaze, medium, and large datasets. This diverse evaluation setup enables a comprehensive assessment of ADCM. As illustrated in Figure 4, the environments used in the experiments are described as follows:

- **HalfCheetah.** A 2D robot consisting of nine links connected by eight joints. The goal is to apply torques to the joints to run forward as fast as possible. The agent receives positive rewards for forward movement and negative rewards for moving backward.
- **Hopper.** A 2D one-legged robot with four main body parts: a torso, a thigh, a leg, and a foot. The goal is to hop forward by applying torques to the three joints connecting these parts.
- **Walker2d.** A 2D bipedal robot consisting of a torso, two thighs, two legs, and two feet. The agent applies torques to six joints to coordinate both legs and walk forward.

Table 1: Performance of ADCM and baselines on Gym tasks. Results for ADCM correspond to the mean normalized scores over 50 random rollouts (5 independently trained models and 10 trajectories per model). The dataset names are abbreviated as follows: medium-expert to "m-e", medium to "m", and medium-replay to "m-r". The best values are marked in bold.

Dataset	BC	CQL	IQL	DT	GDT	CGDT	DC	DM	LSDT	ADCM
halfcheetah-m-e	59.9	91.6	86.7	86.8	92.4	**93.6**	93.0	93.5	93.2	93.3
hopper-m-e	79.6	105.4	91.5	107.6	110.9	107.6	110.4	111.9	111.7	**112.7**
walker2d-m-e	36.6	108.8	109.6	108.1	109.3	109.3	109.6	**111.6**	109.8	111.6
halfcheetah-m	43.1	44.4	47.4	42.6	42.9	43.0	43.0	43.8	43.6	**48.2**
hopper-m	63.9	58.5	66.3	67.6	65.8	96.9	92.5	98.5	87.2	**99.6**
walker2d-m	77.3	72.5	78.3	74.0	77.8	79.1	79.2	80.3	81.0	**87.2**
halfcheetah-m-r	4.3	**45.5**	44.2	36.6	39.9	40.4	41.3	40.8	42.9	45.0
hopper-m-r	27.6	95.0	94.7	82.7	81.6	93.4	94.2	89.1	93.9	**96.7**
walker2d-m-r	36.9	77.2	73.9	66.6	74.8	78.1	76.6	79.3	74.7	**89.1**
avg	46.4	77.6	76.9	74.7	76.8	82.4	81.8	83.2	82.0	**87.0**

Table 2: Performance of ADCM and baselines on Maze2d tasks. Results for ADCM correspond to the mean of normalized scores over 150 random rollouts (5 independently trained models and 30 trajectories per model). The dataset names are abbreviated as follows: umaze to "u", medium to "m", and large to "l". The best values are marked in bold.

Dataset	CQL	DT	QDT	DC	LSDT	ADCM
maze2d-u	94.7	31.0	82.9	10.9	72.3	**125.1**
maze2d-m	41.8	8.2	48.5	16.3	68.4	**128.5**
maze2d-l	49.6	2.3	62.0	20.2	**116.7**	98.7
avg	62.0	13.8	64.5	15.8	85.8	**117.4**

- **Maze2d.** A 2D goal-navigation task in which a force-actuated ball is controlled along the X and Y axes to reach a fixed target location.

Baseline. We compare ADCM with several SOTA offline RL baselines, including the behavior cloning method BC [26]; value-based methods such as CQL [18] and IQL [17]; and RCSL methods including DT [2], GDT [13], QDT [31], CGDT [28], DC [16], DM [21], and LSDT [27]. All methods are evaluated using the standard normalized score = $100 \times \frac{\text{score} - \text{random score}}{\text{expert score} - \text{random score}}$ introduced in [6], which provides a consistent basis for comparing decision-making performance.

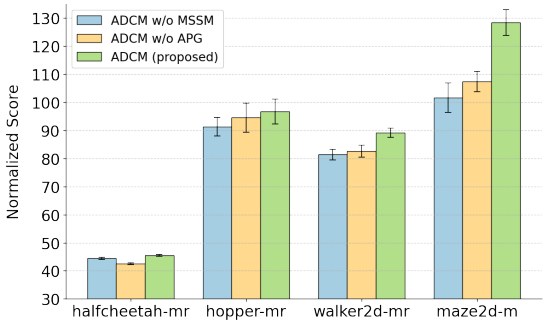


Figure 5: Ablation study. ADCM w/o MSSM denotes ADCM without the multi-scale sequence modeling module, while ADCM w/o APG denotes ADCM without the advantage policy guidance module.

6.2 Overall Performance (RQ1)

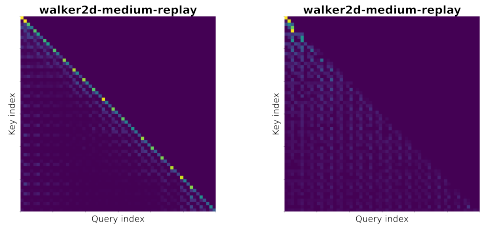
Gym task results. We first evaluate ADCM on the Hopper, Walker2d, and HalfCheetah datasets with Markovian dynamics, with results summarized in Table 1. ADCM achieves SOTA performance across all continuous control tasks, demonstrating the effectiveness of our proposed approach. Notably, ADCM substantially outperforms baseline methods on the medium and medium-replay datasets, indicating that it does not overfit to suboptimal data and substantially improves policy decision quality. Despite leveraging the Q-function from IQL as a reference for advantage action sampling, ADCM still substantially outperforms IQL itself. Overall, ADCM consistently surpasses all baseline methods, highlighting its superior decision-making capability in Markovian environments.

Maze2d task results. To assess the long-term credit assignment and trajectory stitching capabilities of ADCM in non-Markovian environments, we conduct experiments on the Maze2d dataset, with results reported in Table 2. ADCM demonstrates strong performance in these challenging settings. On the umaze and medium

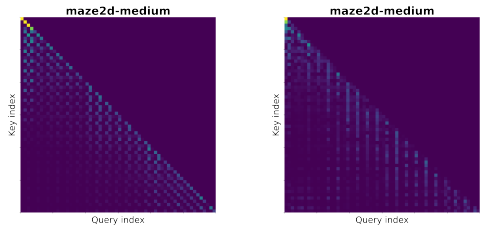
datasets, it significantly outperforms all baseline methods. Although LSDT achieves the highest score on the large dataset—primarily due to its effective goal-conditioning that provides strong guidance in complex environments—ADCM still ranks second with performance close to LSDT. These results confirm that ADCM maintains robust and generalizable decision-making capabilities even on non-Markovian environments.

6.3 Component Ablation Study (RQ2)

To evaluate the contribution of each component in ADCM, we conduct ablation studies on two ADCM variants: one without the multi-scale sequence modeling module (ADCM w/o MSSM) and the other without the advantage policy guidance module (ADCM w/o APG). We perform experiments on three Gym medium-replay datasets and the Maze2d-medium dataset. As shown in Figure 5, removing either component results in a noticeable performance degradation, indicating that both multi-scale sequence modeling and advantage policy guidance are critical to ADCM's effectiveness. Together, these two components enable ADCM to effectively address the core challenges of RCSL.



(a) Walker2d: ADCM vs DT

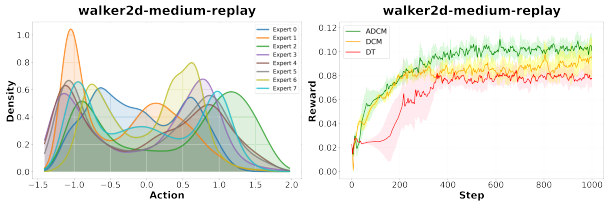


(b) Maze2d: ADCM vs DT

Figure 6: Attention map visualization. For ADCM, a causal self-attention layer is added after the MSSM module to approximate the attention maps, whereas DT employs its final attention layer directly.

6.4 Analysis of Sequence Modeling (RQ3)

To further illustrate the effectiveness of ADCM in modeling complex dependencies, we visualize the dependency patterns captured by ADCM and DT [2] on datasets from the Gym and Maze2d tasks. As shown in the left subfigure of Figure 6a, the attention map of ADCM exhibits a clear diagonal band with a highlighted vertical stripe on the left, indicating that ADCM effectively captures local dependencies with a forgetting mechanism, while also modeling key long-term dependencies. In contrast, the right subfigure of Figure 6a shows that the attention map of DT is relatively uniform, lacking strong focus on the current timestep and selective memory of specific past moments. Figure 6b further shows that in the Maze2d navigation task, both ADCM and DT capture global dependencies, but ADCM attends more precisely to critical long-term dependencies, reflecting a stronger capacity for long-term credit assignment.



(a) Action distribution (b) Action quality

Figure 7: Analysis of action generation in ADCM

6.5 Analysis of Policy Guiding (RQ4)

In this subsection, we explore in depth how the ADCM influences the advantageous action generation of policy.

Does ADCM learn different behavioral patterns within the dataset? In ADCM, we incorporate MoE networks to capture diverse behavioral patterns from the dataset, thereby providing richer action suggestions during inference. To verify whether ADCM effectively learns such patterns, we collect all candidate actions generated by the experts during inference and estimate each expert's action distribution using kernel density estimation. As shown in Figure 7a, the action distributions of the eight experts demonstrate significant diversity, indicating that even under the same state, different experts provide distinct action suggestions. By combining sequential modeling with MoE networks, ADCM successfully learns and represents diverse behavioral patterns, establishing a strong foundation for generating advantageous actions.

Does ADCM guide the policy to generate high-quality actions? To evaluate the quality of actions generated by ADCM, we visualize instantaneous reward variations during inference for ADCM, DT, and DCM (a variant of ADCM without the APG module). As shown in Figure 7b, the instantaneous rewards of DT and DCM remain at lower levels with large fluctuations. In contrast, ADCM achieves higher and more stable instantaneous rewards throughout inference, indicating that it generates higher-quality actions. This improvement is attributed to the diverse action proposals provided by the MoE architecture in ADCM and the effectiveness of group-relative advantage sampling.

Effect of λ on policy guidance in ADCM. Equation (23) shows that our policy guidance mechanism is governed by both return conditioning and group-relative advantage sampling. The parameter $\lambda \in \mathbb{R}$ controls the trade-off between the two: larger values increase the emphasis on group-relative advantage sampling, whereas smaller values emphasize return conditioning. To assess the impact of λ on policy performance, we evaluate ADCM variants with different λ values (i.e., $\lambda \in \{-2, -1, 0, 1, 2\}$) on Gym datasets of

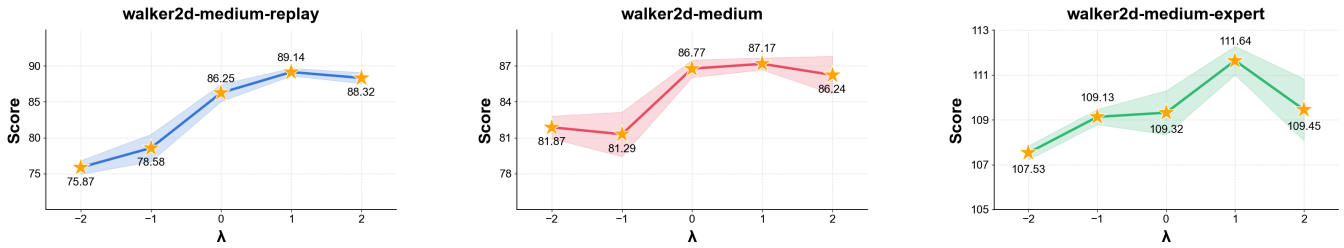


Figure 8: Effect of λ on policy guidance in ADCM. Each dataset corresponds to the mean and standard deviation of the normalized score over 50 random rollouts (5 independently trained models with 10 trajectories per model). The lines represent the changes in the mean values, and the shaded areas indicate the variations in the standard deviation.

Table 3: Hyperparameters used for tasks

Task	Context Length	Target RTG	Training Steps(M_p/M_c)	Number of Experts	λ
HalfCheetah	20	6000,12000	20K / 200k	16	1.5
Hopper	20	3600,7200	50K / 200k	16	0.5
Walker	20	5000,10000	20K / 200k	16	1.0
Maze2d	20	300,600	20K / 200k	16	1.0

Table 4: Model hyperparameters

Hyperparameter	Value
π_{ACDM} lr (learning rate)	0.001
Q_{IQL} lr	0.001
Weight decay	1×10^{-4}
ϵ_{IQL}	0.6
Load factor α	0.05
Discount factor γ	0.99
Optimizer	Adam
Batch size	64
Grad norm clip	0.5
Embedding dimension	128
Activation function	Relu
Dropout	0.1

varying quality. As shown in Figure 8, larger λ values generally yield better decision-making performance across all three datasets, indicating that group-relative advantage sampling serves as a reliable policy guidance mechanism. However, excessive reliance on this mechanism is not always beneficial. When λ becomes too large, performance degrades due to excessive switching between behavioral patterns, which undermines decision coherence and increases error risk. These results underscore the importance of balancing action advantage with behavioral consistency to achieve high-quality policy outputs in ADCM.

6.6 Experiment Setting

Tables 3 and 4 summarize the hyperparameters used in our experiments. The parameters in Table 3 exhibit varying sensitivity across different tasks. Although fine-tuning these parameters could potentially yield marginal performance improvements, we omit an exhaustive hyperparameter search as it falls outside the scope of this work. The hyperparameters in Table 4 correspond to standard settings commonly adopted in deep learning and reinforcement learning. We employ a unified configuration to demonstrate the effectiveness of our method under consistent conditions. All experiments are conducted on an NVIDIA RTX 3090 GPU, where both the critic network and each expert network are instantiated as simple multilayer perceptrons (MLPs).

7 Conclusion

In this work, we propose ADCM to address two major limitations of RCSL—insufficient sequence modeling and limited policy guidance—by introducing two simple yet effective modules. The MSSM module enhances the policy's ability to capture historical interaction trajectories, enabling the extraction of more precise behavioral patterns to support decision-making. Meanwhile, the APG module guides the policy to exploit behavior patterns for high-quality action prediction during inference. Extensive experiments demonstrate the effectiveness of ADCM in the offline RL setting, with promising performance across diverse benchmarks. We expect ADCM to offer new perspectives for advancing research in RCSL. Nevertheless, the APG module still relies heavily on accurate and stable action evaluation. The IQL-based Q-value estimation used in this work remains prone to inaccuracies, which can affect the quality of action sampling. A promising future direction is to explore more precise and robust evaluation methods to further enhance the reliability of advantage-based action sampling.

GenAI Usage Disclosure

ChatGPT (GPT-4) was used only for minor language polishing to improve readability. No generative AI tools were involved in research design, experiments, and content generation. The authors reviewed all text and take full responsibility for the final manuscript.

References

[1] David Brandfonbrener, Alberto Bietti, Jacob Buckman, Romain Laroche, and Joan Bruna. 2022. When does return-conditioned supervised learning work for offline reinforcement learning? *Advances in Neural Information Processing Systems* 35 (2022), 1542–1553.

[2] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Misha Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. 2021. Decision transformer: Reinforcement learning via sequence modeling. *Advances in neural information processing systems* 34 (2021), 15084–15097.

[3] André Correia and Luís A Alexandre. 2023. Hierarchical decision transformer. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 1661–1666.

[4] Tri Dao and Albert Gu. 2024. Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality. arXiv:2405.21060 [cs.LG] https://arxiv.org/abs/2405.21060

[5] Scott Emmons, Benjamin Eysenbach, Ilya Kostrikov, and Sergey Levine. 2021. Rvs: What is essential for offline rl via supervised learning? *arXiv preprint arXiv:2112.10751* (2021).

[6] Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. 2020. D4rl: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219* (2020).

[7] Scott Fujimoto and Shixiang Shane Gu. 2021. A minimalist approach to offline reinforcement learning. *Advances in neural information processing systems* 34 (2021), 20132–20145.

[8] Scott Fujimoto, David Meger, and Doina Precup. 2019. Off-policy deep reinforcement learning without exploration. In *International conference on machine learning*. PMLR, 2052–2062.

[9] Hiroki Furuta, Yutaka Matsuo, and Shixiang Shane Gu. 2021. Generalized decision transformer for offline hindsight information matching. *arXiv preprint arXiv:2111.10364* (2021).

[10] Albert Gu and Tri Dao. 2023. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752* (2023).

[11] Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, et al. 2023. Reinforced self-training (rest) for language modeling. *arXiv preprint arXiv:2308.08998* (2023).

[12] Nico Gürtler, Felix Widmaier, Cansu Sancaktar, Sebastian Blaes, Pavel Kolev, Stefan Bauer, Manuel Wüthrich, Markus Wulfmeier, Martin Riedmiller, Arthur Allshire, et al. 2023. Real robot challenge 2022: Learning dexterous manipulation from offline data in the real world. In *NeurIPS 2022 Competition Track*. PMLR, 133–150.

[13] Shengchao Hu, Li Shen, Ya Zhang, and Dacheng Tao. 2023. Graph decision transformer. *arXiv preprint arXiv:2303.03747* (2023).

[14] Michael Janner, Qiyang Li, and Sergey Levine. 2021. Offline reinforcement learning as one big sequence modeling problem. *Advances in neural information processing systems* 34 (2021), 1273–1286.

[15] Rahul Kidambi, Aravind Rajeswaran, Praneeth Netrapalli, and Thorsten Joachims. 2020. Morel: Model-based offline reinforcement learning. *Advances in neural information processing systems* 33 (2020), 21810–21823.

[16] Jeonghye Kim, Suyoung Lee, Woojun Kim, and Youngchul Sung. 2023. Decision convformer: Local filtering in metaformer is sufficient for decision making. *arXiv preprint arXiv:2310.03022* (2023).

[17] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. 2021. Offline reinforcement learning with implicit q-learning. *arXiv preprint arXiv:2110.06169* (2021).

[18] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. 2020. Conservative q-learning for offline reinforcement learning. *Advances in neural information processing systems* 33 (2020), 1179–1191.

[19] Zenan Li, Fan Nie, Qiao Sun, Fang Da, and Hang Zhao. 2024. Boosting offline reinforcement learning for autonomous driving with hierarchical latent skills. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 18362–18369.

[20] Jianlan Luo, Perry Dong, Jeffrey Wu, Aviral Kumar, Xinyang Geng, and Sergey Levine. 2023. Action-quantized offline reinforcement learning for robotic skill learning. In *Conference on Robot Learning*. PMLR, 1348–1361.

[21] Qi Lv, Xiang Deng, Gongwei Chen, Michael Yu Wang, and Liqiang Nie. 2024. Decision mamba: A multi-grained state space model with self-evolution regularization for offline rl. *Advances in Neural Information Processing Systems* 37 (2024), 22827–22849.

[22] Keiran Paster, Sheila McIlraith, and Jimmy Ba. 2022. You can't count on luck: Why decision transformers and rvs fail in stochastic environments. *Advances in neural information processing systems* 35 (2022), 38966–38979.

[23] Zhenghao Peng, Quanyi Li, Chunxiao Liu, and Bolei Zhou. 2022. Safe driving via expert guided policy optimization. In *Conference on Robot Learning*. PMLR, 1554–1563.

[24] Jinghuan Shang, Kumara Kahatapitiya, Xiang Li, and Michael S Ryoo. 2022. Starformer: Transformer with state-action-reward representations for visual reinforcement learning. In *European conference on computer vision*. Springer, 462–479.

[25] Charlie Snell, Ilya Kostrikov, Yi Su, Mengjiao Yang, and Sergey Levine. 2022. Offline rl for natural language generation with implicit language q learning. *arXiv preprint arXiv:2206.11871* (2022).

[26] Faraz Torabi, Garrett Warnell, and Peter Stone. 2018. Behavioral cloning from observation. *arXiv preprint arXiv:1805.01954* (2018).

[27] Jincheng Wang, Penny Karanasou, Pengyuan Wei, Elia Gatti, Diego Martinez Plasencia, and Dimitrios Kanoulas. 2025. Long-Short Decision Transformer: Bridging Global and Local Dependencies for Generalized Decision-Making. In *The Thirteenth International Conference on Learning Representations*.

[28] Yuanfu Wang, Chao Yang, Ying Wen, Yu Liu, and Yu Qiao. 2024. Critic-guided decision transformer for offline reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 15706–15714.

[29] Yifan Wu, George Tucker, and Ofir Nachum. 2019. Behavior regularized offline reinforcement learning. *arXiv preprint arXiv:1911.11361* (2019).

[30] Yueh-Hua Wu, Xiaolong Wang, and Masashi Hamaya. 2023. Elastic decision transformer. *Advances in neural information processing systems* 36 (2023), 18532–18550.

[31] Taku Yamagata, Ahmed Khalil, and Raul Santos-Rodriguez. 2023. Q-learning decision transformer: Leveraging dynamic programming for conditional sequence modelling in offline rl. In *International Conference on Machine Learning*. PMLR, 38989–39007.

[32] Tianhe Yu, Garrett Thomas, Lantao Yu, Stefano Ermon, James Y Zou, Sergey Levine, Chelsea Finn, and Tengyu Ma. 2020. Mopo: Model-based offline policy optimization. *Advances in Neural Information Processing Systems* 33 (2020), 14129–14142.

[33] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. 2022. ST-MoE: Designing Stable and Transferable Sparse Expert Models. arXiv:2202.08906 [cs.CL] https://arxiv.org/abs/2202.08906